

[Back](#)[Talk to an Expert](#)

# How to Use Python to Calculate the Piotroski F-Score with Intrinio's API

By Nick Pappagiorgio

April 20, 2022



## What is the Piotroski F-Score?

The Piotroski F-Score, named after accountant Joseph Piotroski, utilizes company financial statements and a binary ranking system to reflect the financial strength of a company.

How to Calculate the Piotroski F Score Using Py...



In total there are nine (9) criteria (or conditions) that make up the Piotroski Scoring Scale and a point is awarded for each criteria that satisfies the relevant conditions. These points are then accumulated and the final number, or ranking, sits on a scale between 0 to 9, where 0 indicates a poorly operating company and 9 indicates a profitable, efficient company.

## How is the Piotroski F-Score Used?

Due to the Piotroski F-Score's effectiveness at finding companies that are reducing debt, decreasing the dilution of their shares, and increasing their efficiency, margins and overall profitability, many investors find it a helpful tool to screen for companies to invest in.

Additionally, some investors utilize the criteria on a quarterly basis to check the overall health of their portfolio by tracking the mean Piotroski F-Score and trimming equities below the mean while at the same time allocating funds to companies who score above it.

## What are the Piotroski F-Score Categories?

There are three main categories that make up the Piotroski F-Score for any given company and the accompanying scoring conditions:

- Profitability;
- Leverage, Liquidity, & Source of Funds; and
- Operating Efficiency.

Satisfying a condition outlined below increases the Piotroski F-Score for a company by a point, otherwise the company's Piotroski F-Score remains unchanged.

## Profitability & Leverage

- Positive Net Income in the Current Year
  - Net Income > 0
- Positive Operating Cash Flow in the Current Year
  - Operating Cash Flow > 0
- Positive Change in Return On Assets
  - Current Year ROA > Previous Year ROA
- Accruals in the Current Year
  - Operating Cash Flow > Net Income

## Leverage, Liquidity and Source of Funds

- Reduction of Long-Term Debt
  - Current Year Long-Term Debt < Previous Year Long-Term Debt
- Year over Year Increase in Current Ratio
  - Current Year Current Ratio > Previous Year Current Ratio
- No Share Dilution
  - Current Shares Outstanding <= Previous Year Shares Outstanding

## Operating Efficiency

- Profitability Improvements
  - Current Year Gross Margin > Previous Year Gross Margin
- Asset Turnover
  - Current Year Asset Turnover Ratio > Previous Year Asset Turnover Ratio

## How to Calculate the Piotroski F-Score with Intrinio's API

Utilizing Intrinio's Data Tag System and Standardized Fundamentals API we can easily build out a functional pipeline in Python and quickly query the data required to calculate the Piotroski F-Score for any given company.

### Intrinio Endpoint Used

- [Historical Data For Company](#)

### Intrinio Data Tags Used

- [Net Income](#)
- [Net Cash From Operating Activities](#)
- [Return on Assets](#)
- [Long Term Debt](#)
- [Current Ratio](#)
- [Weighted Average Diluted Shares Outstanding](#)
- [Gross Margin](#)

### Step 1: Retrieve Current and Prior Year Data

We first need to construct a function that retrieves data for the current fiscal year period and the prior fiscal year period. The `_historical_fundy_dataset` function queries the current and prior year data for each data tag passed in as an argument, and returns a dataframe row with the combined current and prior period data.

```
# Data handling
import requests
import numpy as np
import pandas as pd

api_key = "YOUR_API_KEY_HERE"

def _historical_fundy_dataset(ticker, data_tag):
    res = requests.get(f"https://api-v2.intrinio.com/companies/{ticker}/historical_data/{data_tag}?type=FY&api_key={api_key}")
    historical_data = res.json().get("historical_data")
    if len(historical_data) <= 0:
        # Grab only the Most Recent and Prior FY Data
        historical_fundy_dataset = pd.DataFrame(historical_data).set_index("date", inplace=False).rename(columns={"value": f"{data_tag}"})
        historical_fundy_dataset = historical_fundy_dataset.T.iloc[:, 0:2]
        return historical_fundy_dataset
    return pd.DataFrame()
```

## Step 2: Aggregate all requisite data into a single Pandas DataFrame

The second function `_piotroski_dataset` iterates through the requisite data tags needed to perform our conditional checks and aggregates them into a final DataFrame for us to perform our Piotroski conditional calculations on.

```
def _piotroski_dataset(ticker):  
    # Requisite Data Tags  
    data_tag_list = [  
        "netincome",  
        "netcashfromoperatingactivities",  
        "roa",  
        "longtermdebt",  
        "currentratio",  
        "weightedaverage dilutedsharesos",  
        "grossmargin",  
        "assetturnover"  
    ]  
  
    # Compile Latest & Prior Year Fundy Tag Data  
    historical_fundy_datasets = [_historical_fundy_dataset(ticker, data_tag) for data_tag in data_tag_list]  
    piotroski_dataset = pd.concat(historical_fundy_datasets)  
  
    return piotroski_dataset
```

## Step 3: Utilize Piotroski Data to Calculate and Return Overall Piotroski F-Score

Finally, with our Piotroski Dataset complete — filled with the requisite fundamentals — the last function `_piotroski_score` uses the current and prior period financial values for a company to calculate and return the overall aggregated Piotroski F-Score.

Note: In order to prevent returning a “faulty” score, if a company does not have a full fundamental history (perhaps because it recently IPO'd) then you will be unable to calculate the Piotroski F-Score and the function will return None by default.

```
def _piotroski_score(ticker):  
    piotroski_score = None  
    piotroski_dataset = _piotroski_dataset(ticker)  
  
    # Only Process Calc's if Data Available for Each  
    if len(piotroski_dataset) == 8:  
        conditions = (  
            piotroski_dataset.loc["netincome"][0] > 0,  
            piotroski_dataset.loc["netcashfromoperatingactivities"][0] > 0,  
            piotroski_dataset.loc["roa"][0] > piotroski_dataset.loc["roa"][1],  
            piotroski_dataset.loc["netcashfromoperatingactivities"][0] > piotroski_dataset.loc["netincome"][0],  
            piotroski_dataset.loc["longtermdebt"][0] < piotroski_dataset.loc["longtermdebt"][1],  
            piotroski_dataset.loc["currentratio"][0] > piotroski_dataset.loc["currentratio"][1],  
            piotroski_dataset.loc["weightedaverage dilutedsharesos"][0] < piotroski_dataset.loc["weightedaverage dilutedsharesos"][1],  
            piotroski_dataset.loc["grossmargin"][0] > piotroski_dataset.loc["grossmargin"][1],  
            piotroski_dataset.loc["assetturnover"][0] > piotroski_dataset.loc["assetturnover"][1],  
        )  
  
        piotroski_score = sum(conditions)  
  
    return piotroski_score  
  
    piotroski_score = _piotroski_score("SQ") # Score: 5  
    piotroski_score = _piotroski_score("MNDN") # Score: 6  
    piotroski_score = _piotroski_score("TSLA") # Score: 7  
    piotroski_score = _piotroski_score("VZ") # Score: 5  
    piotroski_score = _piotroski_score("PPL") # Score: 5  
    piotroski_score = _piotroski_score("LTX") # Score: 7  
    piotroski_score = _piotroski_score("CSP") # Score: 6
```

## Full Code

```
# Data Handling I  
import requests  
import numpy as np  
import pandas as pd  
  
api_key = "YOUR_API_KEY_HERE"  
  
def _historical_fundy_dataset(ticker, data_tag):  
    res = requests.get(f"https://api-v2.intrinio.com/companies/{ticker}/historical_data/{data_tag}?type=F&api_key={api_key}")  
    historical_data = res.json().get("historical_data")  
  
    if len(historical_data) != 8:  
        # Grab only the Most Recent and Prior FY Data  
        historical_fundy_dataset = pd.DataFrame(historical_data).set_index("date", inplace=False).rename(columns={"value": f"{data_tag}"})  
        historical_fundy_dataset = historical_fundy_dataset.T.iloc[:, 0:2]  
  
    return historical_fundy_dataset  
  
    return pd.DataFrame()  
  
def _piotroski_dataset(ticker):  
    # Requisite Data Tags  
    data_tag_list = [  
        "netincome",  
        "netcashfromoperatingactivities",  
        "roa",  
        "longtermdebt",  
        "currentratio",  
        "weightedaverage dilutedsharesos",  
        "grossmargin",  
        "assetturnover"  
    ]  
  
    # Compile Latest & Prior Year Fundy Tag Data  
    historical_fundy_datasets = [_historical_fundy_dataset(ticker, data_tag) for data_tag in data_tag_list]  
    piotroski_dataset = pd.concat(historical_fundy_datasets)  
  
    return piotroski_dataset  
  
def _piotroski_score(ticker):  
    piotroski_score = None  
    piotroski_dataset = _piotroski_dataset(ticker)  
  
    # Only Process Calc's if Data Available for Each  
    if len(piotroski_dataset) == 8:  
        conditions = (  
            piotroski_dataset.loc["netincome"][0] > 0,  
            piotroski_dataset.loc["netcashfromoperatingactivities"][0] > 0,  
            piotroski_dataset.loc["roa"][0] > piotroski_dataset.loc["roa"][1],  
            piotroski_dataset.loc["netcashfromoperatingactivities"][0] > piotroski_dataset.loc["netincome"][0],  
            piotroski_dataset.loc["longtermdebt"][0] < piotroski_dataset.loc["longtermdebt"][1],  
            piotroski_dataset.loc["currentratio"][0] > piotroski_dataset.loc["currentratio"][1],  
            piotroski_dataset.loc["weightedaverage dilutedsharesos"][0] < piotroski_dataset.loc["weightedaverage dilutedsharesos"][1],  
            piotroski_dataset.loc["grossmargin"][0] > piotroski_dataset.loc["grossmargin"][1],  
            piotroski_dataset.loc["assetturnover"][0] > piotroski_dataset.loc["assetturnover"][1],  
        )  
  
        piotroski_score = sum(conditions)  
  
    return piotroski_score  
  
    piotroski_score = _piotroski_score("SQ") # Score: 5  
    piotroski_score = _piotroski_score("MNDN") # Score: 6  
    piotroski_score = _piotroski_score("TSLA") # Score: 7  
    piotroski_score = _piotroski_score("VZ") # Score: 5  
    piotroski_score = _piotroski_score("PPL") # Score: 5  
    piotroski_score = _piotroski_score("LTX") # Score: 7  
    piotroski_score = _piotroski_score("CSP") # Score: 6
```

## Where can you access reliable financial data?

If you are looking for access to reliable, accurate financial data to power your own financial platforms or models reach out to one of our data specialists today. Intrinio is a leading financial data provider that offers a variety of [packages](#), and we're always eager to assist and guide you in the right direction.



Academic , Data Tools , Developers , Fintech

Share

Tweet

Share

#### Related Posts



December 16, 2021

#### Architecting for Real-Time Stock Data

By Shawn Snyder



April 6, 2022

#### What is Real-Time Market Data & How To Use It

By Briana Christina



March 31, 2022

#### Top Ways Clients Use Intrinio's Data

By Briana Christina

#### Join our mailing list

Subscribe

Intrinio

We can't wait to see what you build

[Pricing & Packages](#)

© Intrinio Inc. 2023

[Privacy Policy](#)

[Terms of Service](#)

